

Document no: P1728r0

Date: 2019-06-16

Authors: Andrzej Krzemieński

Reply-to: akrzemi1 (at) gmail (dot) com

Audience: EWG

Preconditions, axiom-level contracts and assumptions – an in depth study

This paper provides a summary of recent discussions about contracts in the reflector. The goal is to offer a better understanding of the issues and capabilities of the current design. We try to clarify what an “assumption” an “axiom” is, and how the purpose of the contract statement is different from particular semantics associated with it.

Making use of contract declarations

The goal of contract declarations is for the programmer to provide an information about the program. Any contract declaration divides code into two parts: the “before” and the “after”. If the predicate can be determined to be `false` at a given point in time, it means that the code before the contract declaration has a bug. The bug does not have to be *immediately* before contract declaration: it can be far away, but still *before*. Passing an invalid argument to a function is not necessarily a bug itself: it is a symptom of a bug that may be somewhere else.

The information is provided in a formal way, so that not only humans but also automated tools are able to understand it. Thus, contract statements introduce the notion of a *bug* into the language.

Contract declarations can help programmers understand the components they are using and avoid planting bugs in the first place. They can also assist code reviews: correctness can be assessed more easily, and more bugs can be detected by manual inspection.

Static analyzers, including those embedded in the compilers, can make use of contract declarations to detect potential bugs.

Finally, the presence of the additional information can affect how compilers generate the executable code, in a number of ways. This is based on the important semantic effect of contract declarations: if their condition is evaluated to false (or is determined to be false by other means), the compiler is allowed to modify the observable behavior of the program within certain limits.

Contract violation handler

One important semantic effect is that under certain build modes the program is allowed to call the contract violation handler when contract condition is determined to be `false`. It is expected of the handler to have side effects, such as logging the violation. Any side effect is by definition a change in semantics, and any such side effect is allowed for the case where the contract condition is `false` (which is equivalent to proving that program has a bug). In fact, under certain build modes the compiler is *required* to call the violation handler with its side effects.

Abort

Another characteristic side effect, allowed to be injected when contract condition is determined to be `false` is to abort the program execution. It gives us two things. First, the guarantee that the program determined to have a bug will not continue its execution. Second, if the program continues, it means that the condition is true – verified at run-time – and program paths after the contract declaration that are reachable only when the condition is `false` can be eliminated. Such elimination can be performed either by the compiler, as an optimization, or by the programmer: he can deliberately choose to neglect the branch outruled by the precondition.

Behavior change permits

The above code path elimination makes the function body potentially faster, when the preconditions are runtime-checked and cause the program to abort. If we reverse this reasoning, this means that disabling the checks may make the function body slower. We would expect that disabling the run-time checks for contracts should not make the program go slower. Therefore, there is an expectation the same branch elimination inside function body should be allowed even in the situation where contract conditions are not runtime checked. This cannot be strictly called an “optimization” as – apart from making the program faster – it also alters the semantics of the program in the paths declared as buggy. It is equivalent to runtime-checking contract conditions and invoking the custom violation handler containing GCC's `__builtin_unreachable()` inside.

The general mental model behind it is this. Now that we have a tool for unambiguously identifying buggy control paths, we can relax the rules of the abstract machine by requiring the declared semantics for non-buggy paths (or, potentially non-buggy paths), and allowing arbitrary semantic modifications in the buggy paths. This permission can be utilized in a number of ways, not mandated by the language:

1. Refuse to compile programs that inevitably lead to contract violation.
2. Enable bug reporting through UB-sanitizer.
3. Selectively runtime-check contract statements, e.g. only default-level preconditions, as described in [\[P1421R0\]](#).
4. Offer an alternative, more configurable, mechanism for installing custom callbacks for cases where a contract condition is violated.

5. Arbitrarily change the behavior of the program in the buggy path in order to make the non-buggy paths run faster. This is equivalent to runtime-checking the contract predicates and installing the violation handler with GCC's `__builtin_unreachable()`.
6. Do nothing, as though there was no contract statement.

All the above things, however, that a compiler can do with the information in contract declarations is secondary. The primary goal of contract declarations is to provide the information: that a program that caused the contract condition to evaluate to `false` has a bug somewhere before the contract declaration. In other words, the main usefulness of contract declarations is not how they affect the code generation, but what they tell us about the program.

Different meanings of word “assume”

In discussions on contracts words “assume” and “assumption” have a number of different meanings. In this section we try to list them.

Input to static analysis

This is in the context of performing static analysis. Suppose some function `f()`, whose both declaration and definition is seen, but not its usages, is analyzed in order to detect potential bugs:

```
Numeric f(Numeric x, Numeric y)
  [[expects : x >= Numeric::zero()]]
  [[ensures r: r >= Numeric::zero()]]
{
  if (meets_criteria(y))
    return Numeric::one();
  else
    return x;
}
```

Because the function has a postcondition, one thing to check is if this postcondition will be satisfied. We do not know what function `meets_criteria()` does, we only know its declaration:

```
bool meets_criteria(Numeric n); // by-value, wide contract
```

But this information is not necessary to achieve our task. We know that function `f()` either returns 1, or it returns `x`. And of `x` we know that the precondition states that it is non-negative. We do not know if in the real program this `x` will be non-negative. We could get a negative value, but this does not affect the static analysis. Static analysis is only interested in what is stated in the precondition. The outcome of static analysis is, “if preconditions of this function are met on entry, I confirm that this function will meet its postcondition.” That is, there is no claim being made if the precondition is actually satisfied or not.

Thus, making the statement like, “if condition C1 is true, then condition C2 is true also” can be seen as “assuming” that C1 is true. But it is not the same as saying “C1 is true”.

For the lack of a better name, I will call this shade of assumption a correctness-proof-assumption.

runtime-verified conditions

This is the case when a given contract statement is compiled in a mode where the condition is evaluated at run-time and upon false result causes the program to terminate (or an exception to be thrown). In [\[P1429r1\]](#)'s terminology, this is the `check_never_continue` semantic. Inside the body of such function, if the condition from the contract statement appears also in the body of the function, this second appearance is *redundant* and it is perfectly legal, and in fact desired, for the compiler to remove it:

```
// compiled with default build level (or check_never_continue semantic)
Numeric f(Numeric x)
[[expects: x >= Numeric::zero()]]
{
    if (x < Numeric::zero()) // this will be elided
        return Numeric::zero(); // this will be elided

    return some_algo(x);
}
```

We can say that with `check_never_continue` semantic of the contract statement the compiler can *assume* that the condition in the contract statement holds inside the function body. This is analogous to the situation where the condition in the `if`-statement (which is run-time checked) can be elided if it reappears inside the block controlled by the `if`-statement:

```
if (x >= 0)
{
    if (x < 0) // will be elided
        throw invalid_argument{""}; // will be elided
    return sqrt(x);
}
```

We will refer to this as runtime-verified-assumption.

Behavior change permit

This is in the context of the UB-based code transformations. If the compiler can prove that a given code path `p` will inevitably lead to UB in some expression `e`, it is allowed to arbitrarily modify `p`. When describing this permission, we can say that the compiler is allowed to *assume* that evaluating `e` has no UB.

Again, for the lack of a better name, we will refer to it as change-permit-assumption.

Code path omitted by the programmer

This can be illustrated with the following simple example.

```
int f(int * p)
{
    return *p;
}
```

The author of this function chose not to specify what happens if `p` is null. Either he controls all the places where the function is invoked (possible for private member functions or functions in anonymous namespaces) and sees that null value is never passed, or he is confident that all potential callers understand his constraints and that they will obey to these constraints.

Analogies to mathematical axioms

C++ introduces an identifier with special meaning: `axiom`. Some people think it clearly reflects the semantics due to the analogy to mathematical axioms, while other people are confused and draw conclusions that they have a tool for declaring optimization hints. In order to clarify this we have to first explain a bit what axioms are in mathematics, and what an analogy is and how it can help and how it can disturb.

Axioms in mathematics

The most important thing: axioms *do not* state the truth. In fact, no statement in logic or mathematics can be said to be true in the strict sense. Logic offers transformations called inferences that are capable of transforming one postulates (premises) into other postulates (logical consequences). The only guarantee that these inferences offer is this 'conditional' one: *if* the premises are true *then* the logical consequence is true. But there is no guarantee as to whether the premises alone are true, or if the consequences are true. No mathematical theory is ever "true": it can only say, "if premises we started with happen to be true, then this theory is also true."

This way no true postulate can be ever determined, so in order to break this uncertainty dependency we agree that for some small set of postulates we will not require that they be derived from other postulates. We do not know if they are true or not, but we have to live with it in order to do any progress in logic or mathematics. We call these postulates *axioms*.

Thus, there is a philosophical discomfort connected with axioms, relevant for our analogy: we build theorems, theories, our models of the world based on them, but there is no way for us to determine if these axioms are actually true. There are some properties we can determine for a set of axioms – that they are inconsistent or that they are insufficient – but this does not determine their truth or falsehood.

On analogies

Analogies differ in the level of precision and formalism. On the one side of the spectrum we have statements “oh, this reminded me of something I learned in mathematics”; on the other we have the mathematical notion of *relation-preserving isomorphism*. Let’s explain the latter with an example.

We have the red blood cell compatibility table:

donor → recepient	O-	O+	A-	A+	B-	B+	AB-	AB+
O-	ok							
O+	ok	ok						
A-	ok		ok					
A+	ok	ok	ok	ok				
B-	ok				ok			
B+	ok	ok			ok	ok		
AB-	ok		ok		ok		ok	
AB+	ok	ok	ok	ok	ok	ok	ok	ok

We could represent the same as a relation on numbers in binary representation: if for every bit position in *lhs* the bit value is less or equal than the corresponding bit value in *rhs* then the relation returns true:

<i>rhs</i> → <i>lhs</i>	000	001	010	011	100	101	110	111
000	yes							
001	yes	yes						
010	yes		yes					
011	yes	yes	yes	yes				
100	yes				yes			
101	yes	yes			yes	yes		
110	yes		yes		yes		yes	
111	yes	yes	yes	yes	yes	yes	yes	yes

That is, we map “O-” onto 000, “O+” onto 001, and so on, and finally we map relation “recepient with blood type *x* can receive blood from donor with blod type *y*” onto relation “for each bit positon *lhs* has smaller or equal value than *rhs*”.

After this transformation the relation preserves the same “characteristics”: it returns the same values for the blood types as its counterpart for the integral values corresponding to the blood types. This is what we call the relation-preserving isomorphism. The benefit we get from defining it is that if for some reason it is easier for us to learn and understand the properties and relations on integers, we can use the isomorphism to later reason about blood types. Thus, we are not simply saying “`0+`” looks similar to number “`0`”, but we also provide a tool to take our experience and intuition with dealing with numbers and apply it to blood cell types.

Analogies of contracts to mathematical axioms

At least four analogies can be drawn between contract declarations and mathematical axioms.

Any contract statement during static analysis

Any contract statement, regardless of if it is a precondition or a postcondition or an assert, regardless if it has level `default` or `audit` or `axiom`, can be used as an input to static analysis. Such analysis can determine if they lead to situations where one of them would be violated (in such case static analyzer would report a warning that program has a bug). This makes contract conditions analogous to axioms in a logical system, static analysis analogous to logical inferences, and detecting errors analogous to determining if a set of axioms is inconsistent.

This analogy is strong as it includes not only one element (axioms), but also others: inferences and axiom inconsistencies.

Any precondition during static analysis

One can imagine the following sort of static analysis. When analyzing one function, we try to determine that when the declared preconditions of the function are met, all control paths cause the function’s postconditions to be satisfied. In such analysis, any precondition, regardless if it has level `default` or `audit` or `axiom`, is analogous to a mathematical axiom, any postcondition, regardless if it has level `default` or `audit` or `axiom`, is a theorem derived from the axioms, performing the analysis is analogous to making a set of inferences, and verifying that all postconditions are met is analogous to proving a theorem.

This analogy is also strong.

Unchecked contract statements during code generation and execution

The title of this subsection says “checked”, but we actually mean semantics `check_and_terminate` from [\[P1429r1\]](#). Suppose we have the following function:

```
void fun(X const& x) [[expects audit: pred(x)]];
```

We compile and run with contract level *default* and continuation mode *off*. Function `fun` declares that it considers it a bug if it is called with the value of `x` that does not satisfy condition `pred(x)`, and is not

required to guarantee anything if such call actually happens. So, the program relies on the fact that `pred(x)` holds, but at the same time under the current build configuration there is no way to runtime-check if this is actually the case. This discomfort resembles the philosophical discomfort of mathematical axioms: we build theorems based on them, but we cannot determine if they are actually true.

We would have the same discomfort if we built the program with contract level *audit* and continuation mode *on*.

Contract statements not checked during multiple code generation and execution passes

Going back to the above example:

```
void fun(X const& x) [[expects audit: pred(x)]];
```

The condition is not detectable at run-time when compiled with default mode. But at least there exists a mode in which we can compile it where the condition is evaluated. In this mode we will not be able to test the program in real production environment, but at least the precondition can be runtime checked on *some* data. Thus, our check is relied upon in one build/run, and run-time tested in another build/run. In contrast to this, contract conditions with level `axiom` are guaranteed never to be runtime-checked in any build/run. The analogy to mathematical axioms is again by resembling the same philosophical discomfort: the condition is relied upon in *all* executions but not runtime-checked in *any* execution.

The last two analogies are weak, as they only include one element (axiom) and nothing else fits into this analogy: what would be an “inference” here? What would a “theorem” be? Or “axiom inconsistency”?

Preprocessing token `axiom` was chosen to reflect the fourth (weak) analogy with mathematical axioms.

Incorrect and unintended analogy to “absolute truths”

We also have to mention one analogy that was never intended, and is absolutely misleading, however, due to the colloquial meaning of “axiom” in everyday speech, is often employed by humans. In the colloquial sense an “axiom” has a property of being “true”, or representing “truth”. It is more “true” than any other statement that anyone could make. An equivalent of “dogma” with a slightly different emotional baggage.

Under this view, when one sees a declaration containing preprocessing token `axiom`, one is inclined to think that this declaration is equivalent to clang’s `__builtin_assume()` or MSVC’s `__assume()`.

This interpretation is incorrect, as contract declarations – regardless of the level – only declare when a part of program before the declaration contains a bug. They never declare absolute truths about the program state.

Concerns about UB

UB-based optimizations

In today's C++ UB can be used for optimizations. The following example illustrates how an UB inside one function (`f`) can alter the body of another function (`g`):

```
int global_error_count = 0;

void log_error() {
    ++global_error_count; // this has side effect
}                       // but is guaranteed to return normally

int f(int* p)
{
    // implicit assumption: p != nullptr
    return *p;
}

int g(int* p)
{
    if (p == nullptr) // this will get elided
        log_error(); // due to the implicit assumption

    return f(p);
}
```

The author of function `f` has made an assumption: it will never receive a null pointer. As a consequence, the function is written in such a way that passing it a null pointer will be UB. Now, function `g` checks if the pointer is null and if so, it “logs” this fact. However it logs it in a specific way: the function never throws and never stops the program via `std::exit()` or `std::abort()` or similar: it is guaranteed to return normally. (`noexcept` is not needed if the compiler can see the function body.) But then it unconditionally calls `f(p)`. If `p` is null, then the call to `f(p)` will be UB; since compiler is allowed to arbitrarily change the meaning of the program on UB, even prior to the UB event, it can eliminate the `if`-statement altogether: we would only see its effects if the program hit UB the moment later. This removal obviously changes the visible effects of the program, but only in the path that has UB. In the other path it makes the program run faster because no condition in `if`-statement needs to be evaluated. This is sometimes called a “time travel optimization”, and is often found surprising as the effects of UB precede the UB. Clang 6 does perform this optimization in `-O3`.

An important thing to stress here is that this is the present behavior in C++. Function `f()` above has a precondition, even though there is no means to express it. And this precondition causes the body of function `g()` to be altered in a potentially surprising way. If contract statements are added to C++, even with “UB on unchecked failed contracts conditions” semantics, they do not introduce any *new* kind of dangerous transformations: they only make these transformations explicit and more easily detectable.

Unintended program modifications

The permission to arbitrarily changing the buggy paths, especially when it cannot be opted out, rises objections. They are based on the observation that there are classes of bugs that the program can deal

with, or whose adverse effect on the program execution are limited and tolerable. The provision to change these “controlled” bugs into uncontrolled and unpredictable program behavior, can change programs with declared bugs that perform within acceptable limits into programs that exceed these safety limits. The following is an example of a function, taken from [\[P1517R0\]](#), that copes with the bug:

```
void handle_drone(FlightPath *path)
    [[expects LEVEL : path != nullptr]] // for static analysis and test builds
{
    if (path == nullptr)                // for production builds
        throw flight_error{};
    // ...
}
```

The strength of this opposition depends largely on the business domain. Self-operated machines that interact with people require lots of safety measures, whereas video games require unlimited performance and can afford to crash or misbehave. In these domains, where runtime performance benefits are preferred to safety precautions, it is a reasonable course of action to compile the program with dangerous code transformations allowed, perform extensive testing to check if the semantics of the program still meet the requirements, and if no bugs are revealed ship the program. This gives sufficient confidence – obviously, not guarantee – that the program will operate within tolerable limits.

The fact that the behavior for contracts that are not runtime-checked is not defined makes it difficult to write code that both uses contracts and tries to manually handle detected bugs. In function `handle_drone()` above, if the precondition is not runtime-checked (because of the build mode) then the compiler is allowed to elide the `if`-statement in function body. The only way around it would be to define a macro that expands to either a precondition or nothing:

```
#if !defined PRODUCTION_BUILD
#define PRECONDITION(C) [[expects LEVEL : C]]
#else
#define PRECONDITION(C)
#endif

void handle_drone(FlightPath *path)
    PRECONDITION(path != nullptr);
```

This concern could be addressed by requiring another two-state switch of implementations that controls what happens when unchecked contract evaluates to `false`: whether nothing happens or undefined behavior.

However, it should be noted that even if this is fixed, contract statements can still cause arbitrary code modifications in unexpected ways; e.g., when there is an UB in the contract condition itself:

```
void f(X* x) [[expects: x->p()]];

void g(X* x)
{
```

```
if (x == nullptr)
    record_bug();    // can be elided

f(x);
}
```

In the above example, `f()` has an explicit precondition, but it also has an implicit one: that `x` is not null: either the programmer forgot to type it, or he considered it so obvious that it didn't even make sense for him to write it down. If this is compiled in the default build mode, the `if`-statement in function `g()` may be elided.

It should be noted that contract statements do not magically address all problems with program safety. They are expressions and like any other expressions, they can cause bugs, UB, etc..

Acknowledgements

This paper is a summary of long discussions in EWG reflector; it is based on many people's contributions.

Joshua Berne and Ryan McDougall reviewed the paper and suggested numerous improvements.

References

[WD] – Richard Smith, N4800, “Working Draft, Standard for Programming Language C++”.

[P0380r0] – G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, “A Contract Design”.

[P1429r1] – Joshua Berne, John Lakos, “Contracts That Work”.

[P1517r0] – Ryan McDougall, “Contract Requirements for Iterative High-Assurance Systems”.

[P1421r0] – Andrzej Krzemieński, “Assigning semantics to different Contract Checking Statements”.